Vectorization of Neural Networks (Optional)

The ability to **vectorize** neural networks is a primary reason they have scaled so successfully. This approach allows them to leverage parallel computing hardware, like **GPUs**, which are highly optimized for a specific mathematical operation: **matrix multiplication**.

Non-Vectorized vs. Vectorized Implementation

This section contrasts the two ways to implement a forward propagation step for a single layer.

- **Non-Vectorized (Loop-based):**
  - This is the "from-scratch" method seen previously.
  - It uses a `for` **loop** to iterate through each neuron (`j`) in the layer.
  - It computes the `z` and `a` for each neuron **one at a time (sequentially)**.
  - This method is easy to understand but very **slow**.
- **Vectorized (Matrix-based):**
  - This method replaces the entire `for` loop with a single operation.
  - It performs all the neuron computations **at the same time (in parallel)** using matrix multiplication.
  - This is exceptionally **fast** on modern hardware.

The Vectorized Implementation

The code for a `dense` layer is radically simplified.

1. **Compute `Z`:**
   - `Z = np.matmul(A_in, W) + B`
2. **Compute `A_out`:**
   - `A_out = g(Z)` (where `g` is the sigmoid function, applied to every element of the `Z` matrix).

Key Concepts in This Implementation

- **`np.matmul`:** This is the NumPy function for **matrix multiplication**. It is the core of the vectorized implementation, replacing the explicit `for` loop.
- **Key Data Representation Change:**
  - To make matrix multiplication work, **all variables** are now treated as **2D arrays (matrices)**, not 1D vectors.

- **`A_in`** (the input activations, or `x`) is now a 2D array (e.g., a 1x2 matrix: `[[...]]`).
- **`W`** (the weights) is a 2D array (matrix).
- **`B`** (the biases) is also a 2D array (e.g., a 1x3 matrix: `[[...]]`).
- The outputs `Z` and `A_out` will also be 2D arrays (matrices).